

1 Language and semantics

A language is a collection of programs. A program is an *abstract syntax tree* (AST), which describes the hierarchy of constructs. An abstract syntax of a programming language describes the format of abstract syntax trees of programs in this language. Thus, a language is a set of objects, each of which can be constructively manipulated.

The semantics of a language $\mathcal{L}$ is a total map

$$\llbracket \bullet \rrbracket_{\mathcal{L}} : \mathcal{L} \rightarrow \mathcal{D}$$

where $\mathcal{D}$ is some *semantic domain*. The choice of the domain is at our command; for example, for Turing-complete languages $\mathcal{D}$ can be the set of all partially-recursive (computable) functions.

2 Interpreters

In reality, the semantics often is described using *interpreters*:

$$eval : \mathcal{L} \rightarrow \text{Input} \rightarrow \text{Output}$$

where Input and Output are sets of (all possible) inputs and outputs for the programs in the language $\mathcal{L}$. We claim $eval$ to possess the following property

$$\forall p \in \mathcal{L}, \forall x \in \text{Input} : \llbracket p \rrbracket_{\mathcal{L}} x = eval\ p\ x$$

In other words, an interpreter takes a program and its input as arguments, and returns what the program would return, being run on that argument. The equality in the definitional property of an interpreter has to be read “if the right hand side is defined, then the left hand side is defined, too, and their values coincide”, and vice-versa.

Why interpreters are so important? Because they can be written as programs in a *meta-language*, or a language of implementation. For example, if we take $\lambda^a\mathcal{M}^a$ as a language of implementation, then an interpreter of a language $\mathcal{L}$ is some $\lambda^a\mathcal{M}^a$ program $eval$, such that

$$\forall p \in \mathcal{L}, \forall x \in \text{Input} : \llbracket p \rrbracket_{\mathcal{L}} x = \llbracket eval \rrbracket_{\lambda^a\mathcal{M}^a} p\ x$$

How to define $\llbracket \bullet \rrbracket_{\lambda^a\mathcal{M}^a}$? We can write an interpreter in some other language. Thus, a *tower* of meta-languages and interpreters comes into consideration. When to stop? When the meta-language is simple enough for intuitive understanding (in reality: some math-based frameworks like operational, denotational or game semantics, etc.)

Pragmatically: if you have a good implementation of a good programming language you trust, you can write interpreters of other languages.

3 Compilers

A compiler is just a language transformer

$$comp : \mathcal{L} \rightarrow \mathcal{M}$$

for two languages $\mathcal{L}$ and $\mathcal{M}$; we expect a compiler to be total and to possess the following property:

$$\forall p \in \mathcal{L} \quad \llbracket p \rrbracket_{\mathcal{L}} = \llbracket comp\ p \rrbracket_{\mathcal{M}}$$

Again, the equality in this definition is understood functionally. The property itself is called a *complete* (or full) correctness. In reality compilers are *partially* correct, which means, that the domain of compiled programs can be wider.

And, again, we expect compilers to be defined in terms of some implementation language. Thus, a compiler is a program (in, say, $\lambda^a\mathcal{M}^a$), such, that its semantics in $\lambda^a\mathcal{M}^a$ possesses the following property (fill the rest yourself).

4 The first example: the language of expressions

Abstract syntax:

$$\begin{array}{ll} \mathcal{X} &= \{x, y, z, \dots\} & \text{(variables)} \\ \otimes &= \{+, -, \times, /, \%, <, \leq, >, \geq, =, \neq, \vee, \wedge\} & \text{(binary operators)} \\ \mathcal{E} &= \mathcal{X} & \text{(expressions)} \\ &\mathbb{N} \\ &\mathcal{E} \otimes \mathcal{E} \end{array}$$

Semantics of expressions:

- state $\sigma : \mathcal{X} \rightarrow \mathbb{Z}$ assigns values to (some) variables;
- semantics $\llbracket \bullet \rrbracket_{\mathcal{E}}$ assigns each to expression a partial map $\Sigma \rightarrow \mathbb{Z}$, where Σ is the set of all states.

Empty state Λ : undefined for any variable.

Big-step operational semantics is defined via a ternary relation

$$\Rightarrow_{\mathcal{E}} \subseteq \Sigma \times \mathcal{E} \times \mathbb{Z}$$

An expression " $\sigma \xRightarrow[\mathcal{E}]{e} n$ " is informally interpreted as "an evaluation of an expression e in a state σ delivers a value n ".

$$\begin{array}{c} \frac{n \in \mathbb{N}}{\sigma \xRightarrow[\mathcal{E}]{n} n} \quad [\text{CONST}] \\[1em] \frac{x \in \mathcal{X}}{\sigma \xRightarrow[\mathcal{E}]{x} \sigma x} \quad [\text{VAR}] \end{array}$$

$$\frac{\sigma \xRightarrow[\mathcal{E}]{l} x, \sigma \xRightarrow[\mathcal{E}]{r} y}{\sigma \xRightarrow[\mathcal{E}]{l \otimes r} x \oplus y} \quad [\text{BINOP}]$$

$\otimes$	$\oplus$ in $\lambda^a\mathcal{M}^a$
+	+
-	-
$\times$	*
/	/
%	%
<	<
>	>
$\leq$	<=
$\geq$	>=
=	=
$\neq$	!=
$\wedge$	&&
$\vee$	!!

Important observations:

1. " $\Rightarrow_{\mathcal{E}}$ " is defined *compositionally*: the meaning of an expression is defined in terms of meanings of its proper subexpressions.
2. " $\Rightarrow_{\mathcal{E}}$ " is total, since it takes into account all possible ways to deconstruct any expression.
3. " $\Rightarrow_{\mathcal{E}}$ " is deterministic: there is no way to assign different meanings to the same expression, since we deconstruct each expression unambiguously.
4. " $\otimes$ " is an element of language *syntax*, while " $\oplus$ " is its interpretation in the meta-language of semantic description (simpler: in the language of interpreter implementation).
5. This concrete semantics is *strict*: for a binary operator both its arguments are evaluated unconditionally; thus, for example, " $1 \vee x$ " is undefined in empty state.

Having " $\Rightarrow_{\mathcal{E}}$ ", the " $\llbracket \bullet \rrbracket_{\mathcal{E}}$ " can be defined in obvious way:

$$\frac{\sigma \xRightarrow[\mathcal{E}]{e} n}{\llbracket e \rrbracket_{\mathcal{E}} \sigma = n}$$